

Livrable 1

1. Table des matières

- 1. Table des matières
- 2. Contexte et problématique
 - 2.1. Contexte
 - 2.2. Problématique
 - 2.3. Objectifs de l'infrastructure
- 3. Analyse des besoins et des contraintes
 - 3.1. Besoins fonctionnels
 - 3.2. Contraintes techniques
- 4. Principes d'architecture retenus
 - 4.1. Une architecture organisée autour du cycle de vie de la donnée
- 5. Architecture globale de la plateforme
- 6. Description détaillée des composants
 - 6.1. MinIO : stockage objet
 - 6.2. MongoDB : gestion des métadonnées
 - 6.3. PostgreSQL
 - 6.4. MLflow
 - 6.5. Pipelines ETL
 - 6.6. Cluster Apache Spark
 - 6.7. API FastAPI
 - 6.8. Interface utilisateur Streamlit
 - 6.9. Docker Compose
- 7. Flux de données de bout en bout
 - 7.1. Acquisition des données
 - 7.2. Préparation des données
 - 7.3. Expérimentation des modèles
 - 7.4. Déploiement et inférence
 - 7.5. Traçabilité et reproductibilité
 - 7.5.1. Diagramme de séquence simplifié
- 8. Choix technologiques et justification
 - 8.1. Langage de développement
 - 8.2. Gestion des dépendances
 - 8.3. Conteneurisation
 - 8.4. API REST
 - 8.5. Interface utilisateur
 - 8.6. Modèle d'apprentissage
 - 8.7. Gestion du cycle de vie des modèles
 - 8.8. Stockage des données
 - 8.9. Préparation des données
 - 8.10. Qualité logicielle
 - 8.11. Intégration et déploiement continu

- 9. Infrastructure as Code et reproductibilité
 - 9.1. Orchestration de l'infrastructure
 - 9.2. Gestion centralisée de la configuration
 - 9.3. Gestion des dépendances Python
 - 9.4. Intégration continue et livraison continue
- 10. Évolutivité de la plateforme (Spark, MLOps, CI/CD)
 - 10.1. Évolution des pipelines de prétraitement
 - 10.2. Industrialisation du cycle de vie des modèles
- 11. Évolution de l'API
 - 11.1. Déploiement continu
 - 11.2. Perspectives d'évolution

2. Contexte et problématique

2.1. Contexte

La transcription automatique de musique (Automatic Music Transcription – AMT) consiste à convertir un signal audio en une représentation musicale exploitable, telle qu'une partition ou un fichier MIDI. Cette problématique mobilise plusieurs disciplines complémentaires, notamment le traitement du signal, le traitement et l'analyse des données et l'analyse musicale.

Le projet présenté dans ce dossier a pour objectif de développer une solution capable de transcrire automatiquement un enregistrement de guitare monophonique ou polyphonique en fichier MIDI et partition. La qualité d'un tel système repose autant sur les performances du modèle de Machine Learning que sur la maîtrise de l'ensemble du cycle de vie des données ayant permis son apprentissage.

La conception d'un système de transcription ne se limite donc pas au développement d'un modèle prédictif. Elle nécessite la mise en place d'une infrastructure capable de collecter, stocker, transformer et tracer les données utilisées pour l'entraînement, tout en garantissant la reproductibilité des expérimentations et le déploiement maîtrisé du modèle retenu.

Dans ce contexte, une architecture globale a été conçue afin de couvrir l'ensemble des besoins du projet, depuis la préparation des données jusqu'à l'exploitation du modèle au travers d'une application web.

2.2. Problématique

Le développement d'un projet de Data Science présente plusieurs défis techniques.

Les données d'apprentissage proviennent de sources hétérogènes et sont constituées de fichiers audio, d'annotations musicales et de métadonnées associées. Leur préparation nécessite plusieurs étapes de traitement successives (prétraitement audio, extraction de caractéristiques, alignement des annotations et construction des jeux de données d'entraînement), dont les résultats doivent rester reproductibles dans le temps.

Par ailleurs, les expérimentations menées durant la phase de recherche conduisent à entraîner plusieurs modèles de Machine Learning utilisant des architectures différentes. Il devient alors indispensable de conserver l'ensemble des paramètres, métriques et artefacts produits afin de comparer objectivement les performances obtenues et de sélectionner le modèle le plus pertinent.

Enfin, le modèle retenu doit pouvoir être intégré dans une application permettant aux utilisateurs de soumettre un fichier audio et d'obtenir automatiquement les résultats de la transcription. Cette dernière étape implique de disposer d'une infrastructure de déploiement garantissant la portabilité de l'application, la reproductibilité de son environnement d'exécution et l'automatisation des opérations de validation et de mise en production.

La problématique peut ainsi être formulée de la manière suivante :

Comment concevoir une infrastructure de données reproductible, évolutive et industrialisable permettant de gérer l'ensemble du cycle de vie d'un projet de Machine Learning, depuis la préparation des données jusqu'au déploiement d'un modèle de transcription musicale ?

2.3. Objectifs de l'infrastructure

Afin de répondre à cette problématique, l'infrastructure conçue poursuit plusieurs objectifs complémentaires :

- centraliser le stockage des données brutes, des jeux de données construits, des modèles et des artefacts de traitement ;
- assurer la traçabilité des jeux de données et des pipelines de prétraitement afin de garantir la reproductibilité des expérimentations ;
- faciliter l'entraînement, la comparaison et le versionnement des modèles de Machine Learning grâce à une plateforme dédiée au suivi des expérimentations ;
- standardiser l'environnement de développement afin de garantir des conditions d'exécution identiques entre les différents environnements (développement, intégration continue et production) ;
- automatiser les opérations de validation, de construction et de déploiement au moyen d'une chaîne d'intégration et de déploiement continus (CI/CD) ;
- proposer une architecture modulaire permettant de faire évoluer les traitements de préparation des données, notamment par l'intégration future d'un moteur de calcul distribué tel qu'Apache Spark, sans remettre en cause l'organisation globale de la plateforme.

L'ensemble de ces objectifs a conduit à concevoir une architecture reposant sur des composants spécialisés, orchestrés par Docker Compose et entièrement décrits sous forme de code (Infrastructure as Code), afin de garantir la reproductibilité, la maintenabilité et l'évolutivité de la solution.

3. Analyse des besoins et des contraintes

L'objectif du projet est de concevoir une infrastructure complète permettant de développer, entraîner, évaluer puis déployer un modèle d'intelligence artificielle capable de transcrire automatiquement un enregistrement audio de guitare monophonique ou polyphonique en partition musicale (MIDI et partition PDF). Cette infrastructure doit couvrir l'ensemble du cycle de vie de la donnée et du modèle, depuis l'acquisition des données jusqu'à la mise à disposition d'une application exploitable par un utilisateur final.

L'analyse des besoins a conduit à distinguer deux usages principaux :

- un environnement de développement et d'expérimentation destiné au data scientist pour construire les jeux de données, entraîner plusieurs modèles et comparer leurs performances ;
- un environnement de déploiement permettant d'exposer le modèle retenu au travers d'une API REST et d'une interface web.

3.1. Besoins fonctionnels

L'infrastructure doit permettre de :

- centraliser les jeux de données audio et leurs annotations ;
- assurer la traçabilité des traitements appliqués aux données ;
- construire plusieurs versions de jeux d'entraînement à partir de pipelines de prétraitement ;
- expérimenter plusieurs architectures de modèles de deep learning ;
- assurer le suivi des expérimentations (paramètres, métriques, modèles et artefacts) ;
- charger le modèle retenu dans une API REST ;
- proposer une interface web simple permettant de déposer un fichier audio et de récupérer la transcription générée ;
- automatiser les contrôles qualité et le déploiement de l'application.

3.2. Contraintes techniques

Plusieurs contraintes ont orienté les choix d'architecture.

Reproductibilité

L'ensemble des environnements Python est géré avec uv, garantissant une installation déterministe des dépendances grâce au fichier uv.lock. Cette approche permet de reconstruire exactement le même environnement.

Conteneurisation

Tous les composants de la plateforme sont exécutés dans des conteneurs Docker orchestrés par Docker Compose. Cette approche permet :

- d'isoler les différents services ;
- de simplifier le déploiement ;
- d'obtenir un environnement identique en développement et en démonstration.

On peut ainsi reconstruire l'intégralité de l'infrastructure à partir d'une simple commande : `docker compose up -d`.

Séparation des responsabilités

L'architecture a été volontairement découpée en composants indépendants :

- stockage objet (MinIO) ;
- bases de données métier (MongoDB et PostgreSQL) ;
- suivi des expérimentations (MLflow) ;
- API d'inférence (FastAPI) ;
- interface utilisateur (Streamlit).

Cette séparation facilite la maintenance et permet de faire évoluer chaque composant indépendamment.

Traçabilité des données

L'infrastructure doit permettre de connaître précisément :

- l'origine d'un fichier audio ;
- le pipeline de prétraitement appliqué ;
- les jeux de données auxquels appartient chaque échantillon ;
- les modèles ayant été entraînés avec ces données.

Cette traçabilité est assurée conjointement par MongoDB, MinIO et MLflow.

Industrialisation

L'ensemble du code est versionné sous GitHub. L'API d'inférence et l'interface utilisateur sont également contrôlées automatiquement via une chaîne CI/CD comprenant :

- l'exécution des tests unitaires et d'intégration ;
- le contrôle qualité avec Ruff ;
- la vérification statique avec MyPy ;
- la construction d'une image Docker ;
- la publication de cette image dans GitHub Container Registry ;
- le déploiement automatique sur Hugging Face Spaces.

Cette automatisation garantit que seule une version validée du projet peut être déployée.

Évolutivité

L'architecture a été pensée pour évoluer. Ainsi, bien que les pipelines ETL soient actuellement développés en Python, un cluster Spark est déjà intégré à l'infrastructure. Il permettra, dans une future version du projet, de distribuer les traitements de prétraitement audio sur plusieurs nœuds sans modifier l'organisation générale de la plateforme.

4. Principes d'architecture retenus

L'infrastructure a été conçue selon une architecture modulaire distinguant clairement les différentes étapes du cycle de vie d'un projet de science des données : collecte des données, préparation des jeux d'entraînement, expérimentation des modèles, déploiement et exploitation. Cette séparation permet de faire évoluer chaque composant indépendamment tout en garantissant la reproductibilité des traitements.

4.1. Une architecture organisée autour du cycle de vie de la donnée

L'architecture s'articule autour de cinq couches fonctionnelles.

Acquisition et stockage des données

Les données sources (enregistrements audio et annotations musicales) sont stockées dans MinIO, utilisé comme stockage objet compatible S3. Les fichiers sont organisés dans plusieurs buckets correspondant aux différentes étapes du projet :

raw : données brutes (audios et annotations) ; processed : données prétraitées, caractéristiques audio (features) et annotations alignées ; output : réservé aux futures productions de l'API ; mlflow : stockage des artefacts générés par MLflow (modèles, figures, métriques, etc.).

Cette organisation permet de conserver l'historique complet des transformations appliquées aux données tout en évitant la duplication des fichiers.

En parallèle, MongoDB stocke les métadonnées décrivant les pipelines de traitement. Chaque objet manipulé (audio, dataset ou pipeline) possède ainsi une description permettant de garantir la traçabilité des traitements réalisés.

Construction des jeux d'entraînement

Les pipelines ETL sont actuellement développés en Python. Ils réalisent successivement :

- la collecte des données (téléchargement des datasets GuitarSet et IDMT-SMT-Guitar)
- la normalisation et le nettoyage des enregistrements audio ;
- l'extraction des représentations fréquentielles (Constant-Q Transform, ...) ;
- l'alignement des annotations musicales ;
- la génération des jeux d'entraînement.

Les informations produites sont enregistrées dans MongoDB tandis que les fichiers générés sont stockés dans MinIO.

L'infrastructure intègre également un cluster Apache Spark composé d'un nœud maître et de deux nœuds de calcul. Cette partie n'est pas encore exploitée dans la version actuelle mais constitue une évolution prévue afin de distribuer les traitements de prétraitement lorsque les volumes de données deviendront plus importants.

Expérimentation des modèles

Les expérimentations sont réalisées avec MLflow.

Chaque entraînement enregistre automatiquement :

- l'identité du dataset utilisé ;
- les paramètres d'apprentissage ;
- les métriques d'évaluation ;
- les modèles entraînés ;
- les graphiques produits durant les expérimentations.

MLflow s'appuie sur deux composants complémentaires :

- PostgreSQL, utilisé comme base de données du serveur MLflow ;
- MinIO, utilisé pour conserver les artefacts (modèles TensorFlow, figures, jeux de paramètres...).

Cette architecture garantit la reproductibilité complète des expérimentations et facilite la comparaison entre plusieurs versions de modèles.

Déploiement du modèle

Le modèle retenu est exporté depuis MLflow sous la forme d'un modèle TensorFlow (.keras) puis intégré au dépôt Git.

Lors du démarrage de l'API, un ModelManager charge automatiquement :

- le modèle TensorFlow ;

- le scaler utilisé lors de l'entraînement ;
- les métadonnées décrivant le modèle.

Le modèle est ainsi chargé une seule fois au démarrage de l'application puis partagé entre toutes les requêtes, ce qui limite fortement le temps de réponse de l'API.

Exposition du service

La couche d'exploitation repose sur deux composants complémentaires :

- une API REST FastAPI qui expose les services de prédiction ;
- une application Streamlit constituant l'interface utilisateur.

Lorsqu'un utilisateur dépose un fichier audio, la requête suit le pipeline suivant :

- réception du fichier par Streamlit ;
- envoi à l'API REST ;
- prétraitement audio ;
- inférence par le modèle TensorFlow ;
- post-traitement musical ;
- génération des fichiers MIDI, SVG et PDF ;
- restitution des résultats à l'utilisateur.

Cette séparation entre interface graphique et logique métier facilite les évolutions futures et permet à d'autres applications de consommer directement l'API.

Une architecture orientée microservices

L'ensemble des composants est exécuté dans des conteneurs Docker indépendants orchestrés par Docker Compose.

Chaque service possède une responsabilité unique :

Service	Rôle
MinIO	Stockage objet des données et des artefacts
MongoDB	Métadonnées des pipelines et des datasets
PostgreSQL	Données métier
PostgreSQL MLflow	Base de données des expérimentations MLflow
MLflow	Gestion des expériences de machine learning
API FastAPI	Inférence et orchestration des traitements
Streamlit	Interface utilisateur
Spark	Prétraitement distribué (évolution prévue)

Cette organisation limite le couplage entre les composants et facilite leur maintenance ou leur remplacement.

Reproductibilité et industrialisation

La reproductibilité constitue un principe central de l'architecture.

L'environnement Python est standardisé avec uv, qui garantit des versions identiques des dépendances grâce au fichier uv.lock.

L'ensemble des services est défini dans un unique fichier docker-compose.yml, permettant de reconstruire automatiquement la plateforme complète.

Enfin, une chaîne CI/CD GitHub Actions automatise pour l'API d'inférence et l'interface utilisateur :

- l'exécution des tests unitaires et d'intégration ;
- la mesure de la couverture de code ;
- l'analyse statique avec Ruff et MyPy ;
- la construction de l'image Docker ;
- la publication de cette image sur GitHub Container Registry ;
- le déploiement automatique sur Hugging Face Spaces.

Cette automatisation garantit qu'une version déployée a systématiquement satisfait les contrôles qualité définis pour le projet.

Justification des choix d'architecture

Les choix retenus répondent aux objectifs identifiés lors de l'analyse des besoins :

- modularité, grâce à la séparation des responsabilités entre les services ;
- traçabilité, grâce à la combinaison de MongoDB, MinIO et MLflow ;
- reproductibilité, assurée par uv, Docker et GitHub Actions ;
- maintenabilité, par une architecture logicielle clairement découpée ;
- évolutivité, grâce à l'intégration anticipée d'un cluster Spark et à la séparation entre entraînement et inférence.

5. Architecture globale de la plateforme

La figure suivante présente l'architecture globale de la plateforme développée dans le cadre du projet.

ARCHITECTURE GLOBALE DE LA PLATEFORME

Guitaflow – Projet de transcription musicale



L'infrastructure est organisée autour de plusieurs couches fonctionnelles couvrant l'ensemble du cycle de vie d'un projet de science des données, depuis l'acquisition des données jusqu'au déploiement du modèle auprès des utilisateurs.

Sources de données

Les données utilisées pour entraîner les modèles sont constituées de fichiers audio et de leurs annotations musicales. Elles sont importées dans la plateforme puis stockées dans MinIO, qui joue le rôle de stockage objet compatible Amazon S3.

Les objets sont répartis dans plusieurs buckets correspondant aux différentes étapes du traitement :

- raw : données brutes (enregistrements audio et annotations) ;
- processed : données prétraitées, caractéristiques audio et jeux d'entraînement ;
- mlflow : artefacts produits lors des expérimentations ;
- output : espace réservé aux productions futures de l'API.

Cette organisation permet de conserver chaque étape de transformation des données sans altérer les données d'origine.

Gestion des métadonnées

Les informations décrivant les jeux de données et les traitements réalisés sont stockées dans MongoDB et PostgreSQL.

La base de données PostgreSQL conserve :

- Les métadonnées des fichiers audios collectés.

La base de données MongoDB conserve :

- les métadonnées des fichiers audio pré-traités ;
- les métadonnées de représentations fréquentielles extraites ;
- les pipelines de prétraitement appliqués ;
- la composition des jeux d'entraînement.

Cette couche garantit la traçabilité complète des traitements réalisés sur les données.

Préparation des données

Les pipelines ETL sont développés en Python et assurent la préparation des données destinées à l'entraînement.

Ils réalisent successivement :

- la collecte et l'ingestion des données ;
- le prétraitement des signaux audio ;
- l'extraction des caractéristiques (features) ;
- l'alignement des annotations musicales ;
- la constitution des jeux d'entraînement ;
- les références vers les objets stockés dans MinIO.

Les résultats produits sont stockés simultanément dans MinIO et référencés dans MongoDB.

L'architecture prévoit également l'intégration d'un cluster Apache Spark afin de distribuer ces traitements sur plusieurs nœuds lors d'une évolution future de la plateforme.

Expérimentation des modèles

L'entraînement et l'évaluation des modèles sont réalisés avec MLflow.

Chaque expérimentation enregistre automatiquement :

- les paramètres d'apprentissage ;
- les métriques obtenues ;
- les modèles entraînés ;
- les artefacts produits (figures, fichiers de sortie, etc.).

MLflow s'appuie sur :

- une base PostgreSQL dédiée au stockage des expérimentations ;
- MinIO pour conserver les artefacts associés.

Cette organisation facilite la comparaison entre plusieurs architectures de modèles et garantit la reproductibilité des expérimentations.

Déploiement du modèle

Une fois le modèle retenu, celui-ci est exporté depuis MLflow sous la forme d'un modèle TensorFlow (.keras) puis intégré à l'application.

Au démarrage de l'API FastAPI, le composant ModelManager charge automatiquement :

- le modèle TensorFlow ;
- le scaler utilisé pendant l'entraînement ;
- les métadonnées nécessaires à l'inférence.

Le modèle reste ensuite chargé en mémoire pendant toute la durée d'exécution de l'application.

Couche applicative

La couche applicative est composée de deux éléments complémentaires :

- une API REST FastAPI, responsable de l'exécution de la chaîne complète de transcription ;
- une interface Streamlit, permettant à l'utilisateur de déposer un fichier audio et de consulter les résultats générés.

Lors d'une demande de transcription, l'API exécute successivement le prétraitement audio, l'inférence du modèle puis le post-traitement musical avant de retourner les fichiers générés (MIDI, partition PDF et représentations graphiques).

Déploiement et exploitation

L'ensemble des services est conteneurisé avec Docker et orchestré par Docker Compose.

Cette infrastructure peut être reconstruite intégralement à partir du dépôt GitHub grâce au fichier docker-compose.yml, ce qui garantit un environnement identique pour le développement, les démonstrations et l'évaluation du projet.

La qualité logicielle et le déploiement sont automatisés par une chaîne GitHub Actions qui :

- exécute les tests unitaires et d'intégration ;
- contrôle la qualité du code avec Ruff et MyPy ;
- mesure la couverture de tests ;
- construit une image Docker ;
- publie cette image sur GitHub Container Registry ;
- déploie automatiquement l'application sur Hugging Face Spaces.

L'application est ainsi accessible publiquement à l'adresse :

Application déployée : https://huggingface.co/spaces/DamienDESSAUX/M2i_CDSD_Projet_Deployment Dépôt

GitHub : https://github.com/DamienDESSAUX-M2i/M2i_CDSD_Projet

6. Description détaillée des composants

L'infrastructure repose sur plusieurs composants indépendants, chacun assurant une responsabilité clairement identifiée. Cette organisation facilite la maintenance, l'évolutivité et la reproductibilité de la plateforme.

6.1. MinIO : stockage objet

MinIO constitue le système de stockage central de la plateforme. Compatible avec l'API Amazon S3, il permet de stocker les données manipulées tout au long du cycle de vie du projet.

Quatre buckets sont utilisés :

Bucket	Contenu
raw	Enregistrements audio et annotations d'origine
processed	Audios prétraités, caractéristiques audio (features) et jeux d'entraînement
output	Réservé aux futures productions de l'API
mlflow	Artefacts générés par MLflow (modèles, figures, métriques, etc.)

Cette organisation permet de conserver les données brutes intactes tout en assurant la traçabilité des différentes étapes de transformation.

Le service MinIO Init initialise automatiquement ces buckets lors du premier démarrage de la plateforme.

6.2. MongoDB : gestion des métadonnées

MongoDB est utilisée pour stocker les informations décrivant les traitements réalisés sur les données.

La base contient notamment :

- les annotations midi ;
- les métadonnées des fichiers audio pré-traités ;
- les informations relatives aux annotations musicales ;
- les pipelines de prétraitement appliqués ;
- la composition des jeux de données ;
- les références vers les objets stockés dans MinIO.

Ce choix permet de représenter naturellement des documents dont la structure peut évoluer au fil des expérimentations sans nécessiter de migration de schéma.

MongoDB joue ainsi un rôle essentiel dans la traçabilité des traitements réalisés.

6.3. PostgreSQL

Deux instances PostgreSQL sont présentes dans l'infrastructure.

Base métier

La première instance est destinée aux données relationnelles de l'application.

La base de données contient :

- les métadonnées des audios collectés.

Elle permettra de disposer d'un système de gestion de bases relationnelles pour les besoins métier futurs de la plateforme.

Base MLflow

Une seconde instance est dédiée exclusivement au serveur MLflow.

Elle stocke :

- les expériences ;
- les paramètres d'entraînement ;
- les métriques ;
- les exécutions (Runs).

La séparation des deux bases garantit l'indépendance entre les données métier et les données liées aux expérimentations.

6.4. MLflow

MLflow constitue la plateforme de gestion du cycle de vie des modèles.

Chaque entraînement enregistre automatiquement :

- les hyperparamètres ;
- les métriques de performance ;
- les modèles produits ;
- les figures d'analyse ;
- les artefacts générés durant l'expérimentation.

Cette organisation permet de comparer facilement plusieurs architectures de modèles et d'identifier celle présentant les meilleures performances.

Le modèle retenu est ensuite exporté au format TensorFlow (.keras) afin d'être intégré dans l'API d'inférence.

6.5. Pipelines ETL

Les pipelines ETL sont actuellement développés en Python.

Ils assurent successivement :

- la collecte et l'ingestion des données ;
- le nettoyage des enregistrements audio ;
- l'extraction des caractéristiques fréquentielles (Constant-Q Transform) ;
- l'alignement des annotations musicales ;
- la génération des jeux d'entraînement.

Chaque étape produit :

- des objets stockés dans MinIO ;
- des métadonnées enregistrées dans MongoDB.

Cette séparation permet de reconstruire facilement un jeu d'entraînement ou d'identifier précisément les traitements ayant conduit à sa création.

L'architecture prévoit l'évolution de ces pipelines vers Apache Spark afin de paralléliser les traitements lorsque le volume de données augmentera.

6.6. Cluster Apache Spark

Le projet intègre un cluster Spark composé :

- d'un nœud maître (Spark Master) ;
- de deux nœuds de calcul (Spark Workers).

Cette infrastructure est déjà déployée via Docker Compose mais n'est pas encore utilisée par les pipelines ETL.

Elle constitue une évolution planifiée de la plateforme afin de distribuer les traitements de prétraitement audio sur plusieurs nœuds et d'améliorer les performances sur de grands volumes de données.

6.7. API FastAPI

L'API constitue le cœur du système d'inférence.

Développée avec FastAPI, elle expose plusieurs points d'entrée REST permettant notamment :

- de vérifier l'état de santé de l'application ;
- de consulter les informations du modèle chargé ;
- de lancer une transcription audio ;
- de récupérer les fichiers générés (MIDI, partition PDF, représentations graphiques).

Au démarrage de l'application, le composant ModelManager charge :

- le modèle TensorFlow ;
- le scaler utilisé lors de l'entraînement ;
- les métadonnées du modèle.

Le modèle est conservé en mémoire afin d'éviter un rechargement à chaque requête.

L'architecture interne de l'API suit une séparation claire des responsabilités :

- routes HTTP ;
- services métier ;
- composants de traitement audio ;
- modèles Pydantic ;
- gestion centralisée des exceptions ;
- injection de dépendances.

Cette organisation facilite les tests unitaires, la maintenance et les évolutions futures.

6.8. Interface utilisateur Streamlit

Une interface web développée avec Streamlit permet d'utiliser le modèle sans connaissance technique.

L'utilisateur peut :

- déposer un fichier audio au format WAV ;
- lancer automatiquement la transcription ;
- consulter les informations du modèle chargé ;
- récupérer les fichiers produits (MIDI, partition PDF, représentations graphiques).

L'application Streamlit communique exclusivement avec l'API REST, ce qui découple totalement la présentation de la logique métier.

6.9. Docker Compose

L'ensemble de la plateforme est orchestré par un unique fichier `docker-compose.yml`.

Celui-ci permet de démarrer automatiquement :

- MinIO ;
- MongoDB ;
- PostgreSQL ;
- MLflow ;
- Spark ;
- FastAPI ;
- Streamlit.

Les dépendances entre services sont gérées grâce aux health checks et aux mécanismes `depends_on`, garantissant un démarrage cohérent de l'ensemble de la plateforme.

Cette approche permet de reconstruire l'intégralité de l'environnement d'exécution avec une simple commande : `docker compose up -d`.

##5.10 Qualité logicielle et intégration continue

La qualité du projet est assurée par une chaîne CI/CD GitHub Actions.

À chaque modification du dépôt, plusieurs contrôles sont exécutés automatiquement :

- installation reproductible des dépendances avec `uv` ;
- exécution des tests unitaires et d'intégration ;
- calcul du taux de couverture ;
- analyse statique avec `Ruff` ;
- vérification du typage avec `MyPy` ;
- construction d'une image Docker ;
- publication de cette image sur GitHub Container Registry ;
- déploiement automatique sur Hugging Face Spaces.

Cette automatisation garantit qu'une version déployée a satisfait l'ensemble des contrôles de qualité définis pour le projet.

7. Flux de données de bout en bout

L'infrastructure a été conçue pour couvrir l'ensemble du cycle de vie des données, depuis leur acquisition jusqu'à leur exploitation par un utilisateur final. Chaque étape produit des données ou des métadonnées qui sont conservées afin d'assurer la traçabilité des traitements réalisés.

La figure précédente met en évidence les différents composants impliqués dans ce flux.

7.1. Acquisition des données

Le processus débute par la collecte des enregistrements audio de guitare et de leurs annotations musicales.

Ces fichiers sont importés localement puis ingérés dans le bucket raw de MinIO, qui constitue le référentiel des données sources. Les données brutes ne sont jamais modifiées afin de garantir leur intégrité et de permettre la reconstruction complète des traitements.

Parallèlement, les métadonnées associées aux fichiers (identifiant, localisation, informations descriptives, etc.) sont enregistrées dans MongoDB et PostgreSQL.

Cette première étape constitue le point d'entrée de l'ensemble de la chaîne de traitement.

7.2. Préparation des données

Les pipelines ETL exploitent les données présentes dans le bucket raw afin de produire les jeux d'entraînement.

Chaque pipeline réalise successivement :

- le chargement des fichiers audio ;
- le nettoyage et le prétraitement du signal ;
- l'extraction des caractéristiques fréquentielles (Constant-Q Transform) ;
- l'alignement des annotations musicales ;
- la génération des échantillons destinés à l'apprentissage.

Les résultats sont ensuite enregistrés dans le bucket processed de MinIO.

Les métadonnées décrivant les traitements réalisés sont simultanément enregistrées dans MongoDB. Il devient ainsi possible d'identifier précisément les pipelines ayant permis de produire un jeu d'entraînement donné.

Cette organisation garantit une traçabilité complète des transformations appliquées aux données.

7.3. Expérimentation des modèles

Les jeux d'entraînement produits sont utilisés pour entraîner plusieurs modèles de machine learning.

Chaque expérimentation est exécutée au sein de MLflow, qui enregistre automatiquement :

- les hyperparamètres ;
- les métriques d'évaluation ;
- les modèles entraînés ;
- les différents artefacts générés.

Les informations descriptives des expérimentations sont stockées dans PostgreSQL MLflow, tandis que les modèles et les artefacts sont enregistrés dans le bucket mlflow de MinIO.

Cette organisation permet de comparer objectivement plusieurs architectures de modèles et d'assurer la reproductibilité des expérimentations.

À l'issue de cette phase, le modèle offrant les meilleures performances est sélectionné puis exporté au format TensorFlow (.keras) afin d'être utilisé en production.

7.4. Déploiement et inférence

Le modèle sélectionné est intégré à l'API FastAPI.

Lors du démarrage de l'application, le composant ModelManager charge en mémoire :

- le modèle TensorFlow ;
- le scaler utilisé pendant l'entraînement ;
- les métadonnées décrivant le modèle.

Le modèle reste ensuite disponible pour l'ensemble des requêtes sans nécessiter de nouveau chargement.

Lorsqu'un utilisateur utilise l'application, le flux est le suivant :

- dépôt d'un fichier audio WAV depuis l'interface Streamlit ;
- transmission du fichier à l'API REST ;
- prétraitement du signal audio ;
- exécution de l'inférence par le modèle TensorFlow ;
- post-traitement des prédictions (quantification rythmique, reconstruction musicale, génération des partitions) ;
- création des artefacts de sortie (MIDI, SVG, PDF et visualisations) ;
- retour des résultats à l'utilisateur via l'interface web.

L'ensemble de cette chaîne est encapsulé dans le PredictionService, qui orchestre les différents services de prétraitement, d'inférence et de post-traitement.

7.5. Traçabilité et reproductibilité

À chaque étape du flux, les informations nécessaires à la reproductibilité sont conservées.

Les données sources restent disponibles dans MinIO, les traitements appliqués sont enregistrés dans MongoDB et les expérimentations sont historisées dans MLflow.

Le déploiement de l'application est quant à lui automatisé par la chaîne CI/CD, garantissant que le modèle mis à disposition des utilisateurs correspond à une version validée du code source.

Cette organisation permet de reconstruire l'ensemble du processus, depuis les données brutes jusqu'aux résultats produits par l'application, conformément aux exigences de traçabilité et de reproductibilité attendues dans un projet de science des données.

7.5.1. Diagramme de séquence simplifié

Le diagramme ci-dessous illustre le parcours d'un fichier audio depuis son dépôt par l'utilisateur jusqu'à la génération des artefacts musicaux.

```
sequenceDiagram
    autonumber

    actor User as Utilisateur
    participant ST as Streamlit
    participant API as FastAPI
    participant PS as PredictionService
    participant PRE as PreprocessingService
```

```
participant INF as InferenceService
participant POST as PostprocessingService
participant MODEL as Modèle RCNN (.keras)

User->>ST: Dépose un fichier WAV
ST->>API: POST /predict

API->>PS: predict(audio)

PS->>PRE: Prétraitement audio
PRE-->>PS: Spectrogramme CQT

PS->>INF: Inférence
INF->>MODEL: predict()
MODEL-->>INF: Probabilités
INF-->>PS: Notes détectées

PS->>POST: Reconstruction musicale
POST-->>PS: MIDI + SVG + PDF + métriques

PS-->>API: PredictionResponse
API-->>ST: ApiResponse JSON
ST-->>User: Affichage et téléchargement
```

8. Choix technologiques et justification

Les technologies retenues ont été sélectionnées afin de répondre à quatre objectifs principaux :

- garantir la reproductibilité des traitements ;
- faciliter le développement et les expérimentations ;
- permettre le déploiement dans un environnement standardisé ;
- assurer la maintenabilité de la plateforme.

8.1. Langage de développement

L'ensemble du projet est développé en **Python 3.13**, langage de référence en science des données. Son écosystème offre des bibliothèques matures pour le traitement du signal, l'apprentissage automatique, le calcul scientifique et le développement d'API.

8.2. Gestion des dépendances

Le projet utilise **uv** pour gérer les dépendances et l'environnement d'exécution.

Ce choix garantit :

- une installation reproductible grâce au fichier **uv.lock** ;
- des temps d'installation réduits ;
- une gestion centralisée des groupes de dépendances (API, développement, pipelines de données, etc.).

Cette approche permet de reconstruire exactement le même environnement logiciel.

8.3. Conteneurisation

L'ensemble de la plateforme est conteneurisé avec **Docker**.

Chaque composant (API, bases de données, stockage objet, MLflow, Spark...) est isolé dans son propre conteneur afin de :

- simplifier le déploiement ;
- garantir l'isolation des dépendances ;
- reproduire un environnement identique sur tous les postes.

L'orchestration est assurée par **Docker Compose**, qui permet de démarrer l'intégralité de l'infrastructure à l'aide d'une seule commande :

```
docker compose up -d
```

Cette approche facilite également les démonstrations et les phases de développement.

8.4. API REST

L'exposition du modèle est réalisée avec **FastAPI**.

FastAPI a été retenu pour :

- ses performances élevées ;
- sa gestion native des annotations de types Python ;
- l'intégration avec Pydantic pour la validation des données ;
- la génération automatique de la documentation OpenAPI.

L'API est organisée selon une architecture en couches distinguant :

- les routes HTTP ;
- les services métier ;
- les modèles de données ;
- les dépendances injectées ;
- les composants techniques (chargement du modèle, configuration, etc.).

Cette séparation améliore la lisibilité, la maintenabilité et la testabilité du code.

8.5. Interface utilisateur

Une interface Web a été développée avec **Streamlit**.

Elle permet de :

- déposer un fichier audio ;
- lancer la transcription ;
- visualiser les informations de traitement ;
- télécharger les fichiers générés (MIDI, partition, piano roll).

Le choix de Streamlit permet de disposer rapidement d'une interface légère adaptée à la démonstration d'un modèle de science des données sans développer une application frontend complexe.

8.6. Modèle d'apprentissage

Le modèle retenu est développé avec **TensorFlow/Keras**.

L'API ne réalise aucun entraînement.

Au démarrage de l'application :

- le modèle `.keras` est chargé en mémoire ;
- les métadonnées sont lues ;
- le scaler est restauré.

Les prédictions sont ensuite réalisées à partir de cette unique instance afin d'éviter tout rechargement pendant les requêtes.

8.7. Gestion du cycle de vie des modèles

Le suivi des expérimentations est assuré par **MLflow**.

Il permet de conserver :

- les paramètres des expériences ;
- les métriques ;
- les versions des modèles ;
- les artefacts produits.

Le modèle finalement retenu est ensuite extrait manuellement de MLflow afin d'être intégré à l'API de production.

Cette séparation entre expérimentation et déploiement permet de distinguer clairement les environnements de développement et d'exploitation.

8.8. Stockage des données

Plusieurs technologies de stockage sont utilisées selon la nature des données.

MinIO assure le stockage objet compatible S3 :

- audios bruts ;
- audios prétraités ;
- jeux de données ;
- artefacts MLflow.

MongoDB stocke les informations métier nécessaires aux pipelines de préparation des données :

- annotations musicales ;
- métadonnées des traitements appliqués ;
- composition des jeux de données.

Cette approche permet de conserver la traçabilité complète des jeux de données utilisés lors des expérimentations.

Deux bases **PostgreSQL** sont utilisées :

- une première pour les données de l'application ;
- une seconde dédiée exclusivement au backend de MLflow.

Cette séparation évite de mélanger les données métier et les informations liées au suivi des expérimentations.

8.9. Préparation des données

Les pipelines ETL sont actuellement développés en Python.

Une architecture **Apache Spark** est néanmoins intégrée dans la plateforme afin de préparer une évolution vers des traitements distribués lorsque le volume de données augmentera.

Cette évolution concernera principalement les traitements de prétraitement audio.

8.10. Qualité logicielle

La qualité du code est contrôlée automatiquement grâce à :

- **Pytest** pour les tests unitaires et d'intégration de l'API ;
- **Ruff** pour l'analyse statique et le respect des conventions ;
- **MyPy** pour la vérification du typage statique.

Ces contrôles sont exécutés automatiquement dans la chaîne CI/CD avant toute génération d'une image Docker.

8.11. Intégration et déploiement continu

Le projet utilise **GitHub Actions** pour automatiser le cycle de livraison.

Chaque modification déclenche automatiquement :

1. l'installation de l'environnement ;
2. l'exécution des tests ;
3. la mesure de la couverture de code ;
4. les contrôles Ruff et MyPy ;
5. la construction d'une image Docker ;
6. la publication de cette image sur GitHub Container Registry ;
7. le déploiement automatique de l'application sur Hugging Face Spaces.

Cette chaîne garantit qu'une version déployée correspond toujours à un code ayant satisfait les contrôles qualité définis dans le projet.

9. Infrastructure as Code et reproductibilité

L'un des objectifs de cette architecture est de garantir la reproductibilité complète de l'environnement de développement, d'expérimentation et de déploiement. L'ensemble de l'infrastructure est décrit sous forme de

code (Infrastructure as Code), permettant à tout développeur ou évaluateur de reconstruire la plateforme de manière identique.

9.1. Orchestration de l'infrastructure

L'ensemble des services est orchestré à l'aide d'un unique fichier `docker-compose.yml`.

Une simple commande :

```
docker compose up -d
```

déploie automatiquement l'ensemble de la plateforme :

- MinIO et ses buckets d'objets ;
- MongoDB et Mongo Express ;
- PostgreSQL et pgAdmin ;
- PostgreSQL dédié à MLflow ;
- serveur MLflow ;
- cluster Spark (Master + Workers) ;
- API FastAPI ;
- interface utilisateur Streamlit.

Les dépendances entre services sont gérées par les mécanismes `depends_on` associés aux healthchecks, garantissant un ordre de démarrage cohérent.

9.2. Gestion centralisée de la configuration

La configuration est externalisée dans un fichier `.env`.

Celui-ci regroupe notamment :

- les paramètres des bases de données ;
- les identifiants MinIO ;
- les URI des différents services ;
- les variables MLflow ;
- les paramètres de l'API.

Cette approche permet :

- d'éviter le codage en dur des paramètres ;
- de différencier facilement les environnements (développement, démonstration, production) ;
- de sécuriser les informations sensibles.

Un fichier `.env.example` est fourni afin de documenter l'ensemble des variables nécessaires à la reconstruction de l'environnement.

9.3. Gestion des dépendances Python

L'ensemble des dépendances Python est géré avec `uv`, utilisé comme standard de développement du projet.

Les bibliothèques sont décrites dans `pyproject.toml`, tandis que le fichier `uv.lock` verrouille précisément les versions installées.

Cette stratégie garantit :

- la reproductibilité des environnements ;
- la cohérence entre développement local et intégration continue ;
- des installations rapides grâce au mécanisme de cache de `uv`.
- Conteneurisation de l'application

L'API et l'interface utilisateur sont distribuées sous la forme d'une image Docker unique.

Cette image :

- embarque l'ensemble des dépendances Python ;
- contient le modèle TensorFlow exporté depuis MLflow ;
- lance simultanément FastAPI et Streamlit via le script `start.sh`.

La conteneurisation permet d'obtenir un comportement identique quel que soit l'environnement d'exécution (poste développeur, GitHub Actions ou Hugging Face).

9.4. Intégration continue et livraison continue

Le projet est automatisé au moyen d'une pipeline GitHub Actions.

À chaque modification de la branche principale :

- les dépendances sont installées avec `uv` ;
- les tests unitaires et d'intégration sont exécutés ;
- le taux de couverture est vérifié ;
- la qualité du code est contrôlée avec `Ruff` ;
- le typage statique est validé avec `MyPy` ;
- une image Docker est construite puis publiée sur GitHub Container Registry (GHCR) ;
- le déploiement sur Hugging Face Spaces est réalisé automatiquement en référençant cette image.

Cette automatisation réduit les opérations manuelles, limite les risques d'erreur et garantit que toute version déployée a été préalablement validée.

10. Évolutivité de la plateforme (Spark, MLOps, CI/CD)

L'architecture a été conçue selon une approche modulaire afin de faciliter son évolution sans remettre en cause les composants existants.

10.1. Évolution des pipelines de prétraitement

À ce jour, les pipelines ETL sont implémentés en Python.

Afin d'anticiper une augmentation du volume de données, un cluster Apache Spark est déjà intégré à l'infrastructure Docker Compose.

Même si cette partie n'est pas encore exploitée en production, l'objectif est de migrer progressivement les traitements de prétraitement audio vers Spark afin de bénéficier :

- d'une meilleure parallélisation ;
- d'une montée en charge simplifiée ;
- d'une réduction des temps de traitement sur de grands corpus audio.

L'intégration de Spark a donc été anticipée dès la phase de conception.

10.2. Industrialisation du cycle de vie des modèles

La plateforme sépare clairement :

- la phase d'expérimentation des modèles ;
- la phase d'inférence.

Les expérimentations sont réalisées via MLflow, qui assure :

- le suivi des métriques ;
- l'historisation des paramètres ;
- le stockage des modèles ;
- la conservation des artefacts.

L'API n'entraîne aucun modèle.

Elle charge uniquement un modèle TensorFlow (.keras) exporté depuis MLflow et validé en amont.

Cette séparation garantit une meilleure stabilité du service de prédiction et facilite le remplacement futur du modèle sans modifier le code métier.

11. Évolution de l'API

L'API FastAPI repose sur une architecture en couches distinguant :

- les routes HTTP ;
- les services métier ;
- les composants de traitement audio ;
- le gestionnaire de modèles ;
- les modèles Pydantic.

Cette organisation facilite :

- l'ajout de nouveaux endpoints ;
- l'intégration de nouveaux modèles ;
- l'évolution des traitements de post-traitement ;
- l'amélioration progressive des algorithmes.

Les dépendances étant injectées au démarrage de l'application, leur remplacement reste localisé et n'impacte pas les routes.

11.1. Déploiement continu

La chaîne CI/CD permet de diffuser automatiquement toute nouvelle version validée.

Après exécution des contrôles qualité, une image Docker est :

- construite ;
- versionnée ;
- publiée sur GitHub Container Registry.

Le déploiement sur Hugging Face Spaces est ensuite déclenché automatiquement en référençant cette image.

Cette approche présente plusieurs avantages :

- une version unique de référence ;
- une parfaite cohérence entre les environnements ;
- un déploiement rapide ;
- une traçabilité complète des versions.

11.2. Perspectives d'évolution

L'architecture retenue ouvre plusieurs perspectives d'amélioration :

- migration complète des traitements ETL vers Apache Spark ;
- exploitation du bucket output de MinIO pour stocker les résultats générés par l'API ;
- automatisation de l'extraction du meilleur modèle depuis MLflow vers l'API ;
- ajout de tests d'intégration de bout en bout sur l'ensemble de la plateforme ;
- mise en place d'une stratégie de monitoring et d'observabilité des services.

Ces évolutions pourront être intégrées progressivement sans remise en cause de l'architecture actuelle grâce à la séparation des responsabilités entre les différents composants.